

Практическая теория типов

Лекция 11: Унивалентность, эквивалентности и высшие индуктивные типы

Воронов Михаил Сергеевич

ВМК МГУ

Осень 2025

План лекции

«Эпилог: типы как пространства и univalent foundations»

- 1 Рекап: Id-типы и h-уровни
- 2 Эквивалентности типов
- 3 Аксиома унивалентности
- 4 Высшие индуктивные типы (HIT)
- 5 Модели и Cubical Type Theory
- 6 Заключение

План лекции

- 1 Рекап: Id-типы и h-уровни
- 2 Эквивалентности типов
- 3 Аксиома унивалентности
- 4 Высшие индуктивные типы (HIT)
- 5 Модели и Cubical Type Theory
- 6 Заключение

Краткое напоминание: Id-типы и пути

Определение (Identity-тип)

Для $a, b : A$ тип путей: $a =_A b$

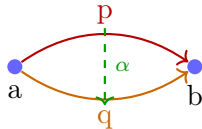
Конструктор: $\text{refl}_a : a = a$

Операции на путях

$p \cdot q$ (конкатенация), p^{-1} (обратный), законы группоида до гомотопии.

Высшие пути

1-пути $p : a = b$, 2-пути $\alpha : p = q$, ... Тип A — ∞ -группоид.



Ключевая идея:

Равенство — это путь, а не примитив.

Напоминание: h -уровни (из лекции 10)

Индуктивное определение

База: $\text{isContr}(A) := \sum_{c:A} \prod_{x:A} (c = x)$ ($h\text{-level } -2$)

Шаг: $h\text{-level } (n + 1) \Leftrightarrow$ типы путей имеют $h\text{-level } n$

Стандартные названия

$h = -2$: contractible; $h = -1$: mere prop; $h = 0$: $h\text{-set}$; $h \geq 1$: группоиды

Фокус лекции

Сегодня про $h\text{-set}$ — уровень, на котором живёт обычная математика.

h-set: интуиция

Что значит $\text{isSet}(A)$?

$$\text{isSet}(A) := \forall(x\ y : A), \text{isProp}(x = y)$$

Для любых $x, y : A$ тип равенств $x = y$ является mere proposition.

Геометрическая интерпретация

- h-set — это тип без нетривиальной топологии
- Между двумя точками либо нет путей, либо есть ровно один (с точностью до гомотопии)
- Нет «петель», которые нельзя стянуть в точку

Эквивалентность с UIP (напоминание)

$$\text{isSet}(A) \iff \text{UIP}_A \iff K_A$$

Все три условия эквивалентны (теорема из лекции 10).

Теорема Хедберга (напоминание)

Теорема (Hedberg, 1998)

Если равенство на типе A разрешимо, то A является h-set:

$$(\forall x y : A, (x = y) + (x \neq y)) \Rightarrow \text{isSet}(A)$$

Следствия

- Bool, $\mathbb{N}$, конечные типы — автоматически h-sets
- Любой тип с decidable equality является h-set
- Доказательство использует «стабильность» функции сравнения

Практическое значение

Для большинства типов данных в программировании равенство разрешимо, поэтому они автоматически являются h-sets без дополнительных доказательств.

Замкнутость h-set относительно конструкторов

Основные результаты

Если A и B — h-set, то:

- $A \times B$ — h-set (равенство пар покомпонентно)
- $A + B$ — h-set (элементы из разных инъекций не равны)
- $\text{List } A$ — h-set (равенство списков поэлементно)
- $A \rightarrow B$ — h-set (при наличии funext; поточечное равенство)

Формально

$$\text{isSet}(A), \text{isSet}(B) \Rightarrow \text{isSet}(A \times B), \text{isSet}(A + B), \text{isSet}(A \rightarrow B)$$

Практически все типы данных в программировании (строки, деревья, записи, ADT) являются h-sets, если их базовые компоненты — h-sets.

h-set в контексте HoTT

Совместимость с унивалентностью

- Глобальный UIP (для всех типов) несовместим с унивалентностью
- Локальный UIP (для конкретного типа A) — допустим
- Большинство «обычных» типов ($\mathbb{N}$, Bool , списки) — h-sets

Типы, не являющиеся h-set

- Вселенная типов $\mathcal{U}$ (унивалентность даёт нетривиальные пути)
- Высшие индуктивные типы с path-конструкторами (окружность $\mathbb{S}^1$, сфера)
- Группоиды, категории как типы

Вывод

h-sets — «безопасная зона», где равенство ведёт себя классически; выход за её пределы требует работы с высшими путями.

Не-пример: $\mathbb{S}^1$ не является h-set

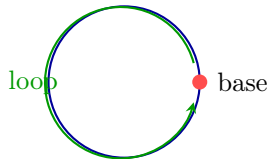
Окружность (HIT)

$\text{base} : \mathbb{S}^1$ — точка; $\text{loop} : \text{base} = \text{base}$ — петля

Почему $\mathbb{S}^1$ не h-set?

- Есть loop^n для всех $n \in \mathbb{Z}$
- $(\text{base} = \text{base}) \simeq \mathbb{Z}$ — бесконечно много путей
- Для isSet нужно $\text{isProp}(\text{base} = \text{base})$
- Но тип $\text{base} = \text{base}$ не proposition

Вывод: $\neg \text{isSet}(\mathbb{S}^1)$ — окружность несёт нетривиальную топологию.



$$\text{loop}^n \neq \text{loop}^m$$

План лекции

- 1 Рекап: Id-типы и h-уровни
- 2 Эквивалентности типов
- 3 Аксиома унивалентности
- 4 Высшие индуктивные типы (HIT)
- 5 Модели и Cubical Type Theory
- 6 Заключение

Равенство vs изоморфизм в обычной математике

Классическая теория множеств

- $A = B$ означает буквально совпадение элементов
- $A \cong B$ («биекция») означает, что множества структурно одинаковы, но не равны

Пример

$\mathbb{N}$ и $\mathbb{Z}$ — не равны как множества, но биективно связаны с $\mathbb{N} \times \text{Bool}$.

$\{0, 1\}$ и $\{\text{чёт, нечёт}\}$ — разные множества, но изоморфны.

Проблема

В теории типов хочется ближе к идее:

«Структурно одинаковые типы — это одно и то же.»

Определение эквивалентности типов

Определение (Эквивалентность в HoTT)

$\text{isEquiv}(f) := \Sigma_{g:B \rightarrow A} ((g \circ f \sim \text{id}_A) \times (f \circ g \sim \text{id}_B))$

$\text{Equiv}(A, B) := \Sigma_{f:A \rightarrow B} \text{isEquiv}(f)$ Обозначение: $A \simeq B$

Пояснения

- $\sim$ — гомотопия (поточечное равенство)
- Эквивалентность — биекция + данные обратимости до гомотопии

Основные свойства $\text{Equiv}(A, B)$

Эквивалентность — это отношение эквивалентности

- Рефлексивность: $\text{id}_A : \text{Equiv}(A, A)$
- Симметрия: из $e : \text{Equiv}(A, B)$ получаем $e^{-1} : \text{Equiv}(B, A)$
- Транзитивность: $\text{Equiv}(A, B)$ и $\text{Equiv}(B, C)$ влечёт $\text{Equiv}(A, C)$

Но есть нюанс

Equiv задаёт отношение эквивалентности на типах,
но пока оно не то же самое, что $\text{Id}_{\mathcal{U}}(A, B)$ (равенство типов).

Как связать эквивалентность и равенство типов?

Конкретные примеры $\text{Equiv}(A, B)$

- ❶ $\text{Bool} \simeq \text{Fin } 2$
 - Явное соответствие: $\text{true} \leftrightarrow 0$, $\text{false} \leftrightarrow 1$
- ❷ $A \times 1 \simeq A$
 - Прямое: $(a, \text{tt}) \mapsto a$
 - Обратное: $a \mapsto (a, \text{tt})$
- ❸ $A + 0 \simeq A$
- ❹ $A \times B \simeq B \times A$ (перестановка компонент пары)
- ❺ $(A \rightarrow B \rightarrow C) \simeq (A \times B \rightarrow C)$ (каррирование)

В HoTT $\text{Equiv}(A \times 1, A)$ — это реальный тип, не просто метаутверждение.

Гомотопия функций

Определение (Гомотопия)

Функции $f, g : A \rightarrow B$ гомотопны: $f \sim g := \forall x, f(x) = g(x)$ (поточечное равенство).

Зачем гомотопия?

Равенство $f = g$ требует `funext`; гомотопия — всегда определена. В HoTT с UA: $f \sim g \Leftrightarrow f = g$.

Пример

$\text{id} \circ f \sim f$ — тривиальная гомотопия: $\text{refl}_{f(x)}$ для каждого x .

Альтернативные определения эквивалентности

Проблема с наивным определением

Наивное « $g \circ f = \text{id}$ и $f \circ g = \text{id}$ » недостаточно: $\text{isEquiv}(f)$ может быть не proposition.

Корректные определения (эквивалентные)

- ❶ Bi-invertible: $\Sigma_g (g \circ f \sim \text{id}) \times \Sigma_h (f \circ h \sim \text{id})$
- ❷ Half-adjoint: один обратный + когерентность
- ❸ Contractible fibers: все слои $f^{-1}(b)$ контрактивны

Важно

Все определения эквивалентны, и $\text{isEquiv}(f)$ — mere proposition.

Каноническая функция idtoEquiv

Связь равенства и эквивалентности

В MLTT можно построить: $\text{idtoEquiv} : (A =_{\mathcal{U}} B) \rightarrow \text{Equiv}(A, B)$

Как строится?

По J-элиминатору: $\text{refl}_A : A = A$ отображается в $\text{id}_A : \text{Equiv}(A, A)$. Любой путь $p : A = B$ транспортируется в эквивалентность.

Ключевой вопрос

idtoEquiv всегда существует. Но является ли она сама эквивалентностью? Можем ли из $\text{Equiv}(A, B)$ получить $A = B$?

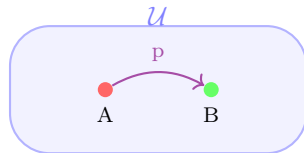
План лекции

- 1 Рекап: Id-типы и h-уровни
- 2 Эквивалентности типов
- 3 **Аксиома унивалентности**
- 4 Высшие индуктивные типы (HIT)
- 5 Модели и Cubical Type Theory
- 6 Заключение

Вселенная $\mathcal{U}$ и пути между типами

Вселенная типов

- $\mathcal{U}$ — тип всех «малых» типов
- $A : \mathcal{U}$ означает « A — тип»
- $\text{Id}_{\mathcal{U}}(A, B)$ — пути между типами



Геометрическая интуиция

$\mathcal{U}$ — «пространство всех типов», путь $p : A = B$ — связь между типами.

Формулировка аксиомы унивалентности

Определение (Univalence Axiom — UA)

$$\text{univalence} : \prod_{A,B:\mathcal{U}} \text{isEquiv}(\text{idtoEquiv}_{A,B})$$

Эквивалентная формулировка:

$$\boxed{\text{Equiv}(A, B) \simeq (A =_{\mathcal{U}} B)}$$

Словами

Каноническая карта idtoEquiv из путей между типами в эквивалентности типов сама является эквивалентностью.

Главный слоган HoTT

«Равенство типов $\equiv$ их эквивалентность»

Интуиция: «equivalent = equal»

Что даёт унивалентность?

В унивалентных основаниях:

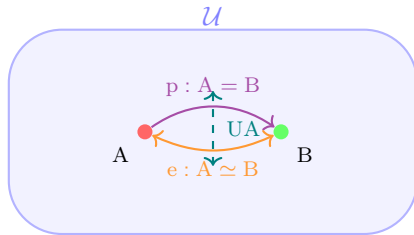
- Два типа считаются «тем же типом», если они эквивалентны
- Это формализует распространённый в математике стиль:

«Мы не различаем изоморфные структуры.»

Примеры из математики

- «Все конечные поля из 4 элементов — одно и то же»
- « $\mathbb{R}^2$ и $\mathbb{C}$ как вещественные пространства — одно»
- «Группа $\mathbb{Z}/2\mathbb{Z}$ и $\{1, -1\}$ — одно и то же»

УА делает это законным на уровне теории типов.



UA говорит

Каждой $e : A \simeq B$ соответствует $ua(e) : A = B$; каждому пути — эквивалентность (через $idtoEquiv$); соответствия взаимно обратны.

Transport по эквивалентностям

Transport в MLTT

Для семейства $P : \mathcal{U} \rightarrow \mathcal{U}$ и пути $p : A = B$: $\text{transport}_P(p) : P(A) \rightarrow P(B)$

С унивалентностью

Эквивалентность $e : \text{Equiv}(A, B)$ даёт путь $\text{ua}(e) : A = B$, по которому можно делать transport .

Мораль

Эквивалентные типы взаимозаменяемы. Если $A \simeq B$, то везде, где стоит A , можно подставить B .

Пример: transport по $A \times 1 \simeq A$

Конкретный пример

Эквивалентность $e : \text{Equiv}(A \times 1, A)$ даёт путь $ua(e) : A \times 1 = A$.

Для $P(X) = \text{List } X$: $\text{transport}_{\text{List}}(ua(e)) : \text{List}(A \times 1) \rightarrow \text{List } A$

Интуитивно

Это функция, превращающая список пар (a, tt) в список элементов a . Transport автоматически переносит данные между эквивалентными типами.

Следствие UA: function extensionality

Напоминание: funext

$\text{funext} : (\forall x, f(x) = g(x)) \rightarrow f = g$ — две функции равны, если равны поточечно.

Статус в разных системах

- В чистой MLTT: добавляется как аксиома
- В HoTT с UA: выводится из унивалентности

Идея доказательства

Функции $A \rightarrow B \approx$ семейства типов $\rightarrow$ поточечное равенство даёт эквивалентность $\rightarrow$ UA превращает в путь.

Почему UA важна?

Практические следствия

- 1 Переписывание по изоморфизму: доказали теорему для $\mathbb{N}$? Автоматически верна для любого типа $\simeq \mathbb{N}$
- 2 Абстракция: реализация не важна, важен только интерфейс (до эквивалентности)
- 3 Function extensionality: не нужна как отдельная аксиома
- 4 Structure identity principle: изоморфные структуры идентичны

Философское значение

UA формализует математическую практику:

«Работаем с объектами с точностью до изоморфизма»

Structure Identity Principle

Идея

Если две математические структуры изоморфны, они равны.

Пример: моноиды

Моноид = $(M, e, \cdot)$. Изоморфизм: $f : M_1 \simeq M_2$ с $f(e_1) = e_2$ и $f(x \cdot y) = f(x) \cdot f(y)$.

С UA: изоморфные моноиды равны как элементы типа моноидов.

Следствие

Теорема для одного моноида автоматически верна для всех изоморфных.

UA и вычисления: проблема

Вычислительное поведение

Ключевое правило UA:

$$\text{transport}(\text{ua}(e), x) = e(x)$$

«Transport по пути, построенному из эквивалентности e , применяет e к аргументу.»

Проблема в классической MLTT + UA

- Если UA — просто аксиома, $\text{transport}(\text{ua}(e), x)$ не редуцируется
- Застревает как «нейтральный терм»
- Ломает каноничность: не можем вычислить $\mathbb{N}$ -терм до numeral

Решение

Нужна система, где UA — не аксиома, а конструктивное правило.

⇒ Cubical Type Theory

План лекции

- 1 Рекап: Id-типы и h-уровни
- 2 Эквивалентности типов
- 3 Аксиома унивалентности
- 4 Высшие индуктивные типы (HIT)**
- 5 Модели и Cubical Type Theory
- 6 Заключение

Обычные индуктивные типы: только точки

Стандартный индуктивный тип

```
data Nat : Type where  
  zero : Nat  
  succ : Nat → Nat
```

Что порождают конструкторы?

- Конструкторы порождают точки (элементы типа)
- Пути между точками появляются только из общих правил Id-типа
- Мы их явно не прописываем

Ограничение

Нельзя напрямую задать тип с «нетривиальной топологией» — например, окружность.

Идея высших индуктивных типов

Определение (Higher Inductive Type — HIT)

HIT позволяют указать: point-конструкторы (точки), path-конструкторы (пути), higher paths (пути между путями).

Что это даёт?

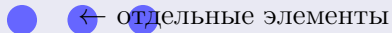
Можно «защить» топологию: $\mathbb{S}^1$ (точка + петля), $\mathbb{S}^2$ (точка + 2-петля), тор...

Синтетическая топология

Топологические пространства как типы — без подлежащего множества точек.

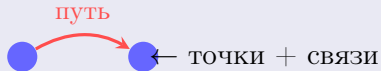
Аналогия: конструктор LEGO для пространств

Обычные индуктивные типы — «точки»



Как набор бусин — есть элементы, но нет связей между ними.

ΠТ — «точки + клей»



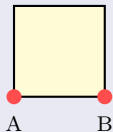
Как конструктор — можем склеивать точки, создавая форму.

Ключевая идея

Path-конструктор — это инструкция по склейке: «точка a и точка b на самом деле одно и то же».

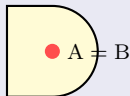
Аналогия: оригами из типов

Лист бумаги (тип)



Есть точки A и B на краях.

Склейка (path-constructor)



Добавляем путь $A = B$ — склеиваем края.

НПТ = правила складывания

- Склеить два конца отрезка $\rightarrow$ получить окружность
- Склеить все края квадрата в точку $\rightarrow$ получить сферу
- Path-конструкторы — это «линии сгиба»

Интервал II: определение

Определение (Интервал как HIT)

```
data I : Type where
  0_I : I; 1_I : I    -- endpoints
  seg : 0_I = 1_I     -- path
```



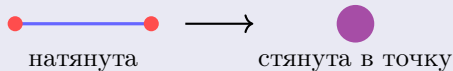
Аналогия: отрезок $[0, 1]$, но без «внутренних точек» — только концы и путь.

Три ингредиента

- ❶ Точка 0_I — «левый конец»
- ❷ Точка 1_I — «правый конец»
- ❸ Путь seg — «сам отрезок»

Интервал: интуиция и свойства

Аналогия: резиновая нить



Интервал можно непрерывно «стянуть» в одну точку — он контрактивен.

Почему $\mathbb{I}$ контрактивен?

- Есть путь $\text{seg} : 0_{\mathbb{I}} = 1_{\mathbb{I}}$ — концы «склеены»
- Любой элемент $\mathbb{I}$ равен $0_{\mathbb{I}}$ (или $1_{\mathbb{I}}$)
- Нет «дырок» или «петель» — топологически тривиален

h-level интервала

$\mathbb{I}$ имеет h-level -2 (contractible) — самый «простой» тип.

Зачем нужен интервал?

Интервал как «параметр пути»

В обычной топологии путь — это функция $\gamma : [0, 1] \rightarrow X$.

В НоТТ путь $p : a = b$ можно представить как «функцию из интервала»:

$$p : \mathbb{I} \rightarrow A \quad \text{с } p(0_I) = a, \quad p(1_I) = b$$

Применения

- Cubical Type Theory: интервал $\mathbb{I}$ — базовый примитив
- Function extensionality: доказывается через интервал
- Построение путей: «провести путь» = задать функцию из $\mathbb{I}$

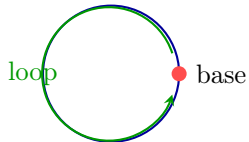
Аналогия: параметр анимации

$\mathbb{I}$ — как параметр $t \in [0, 1]$ в анимации: $t = 0$ — начало, $t = 1$ — конец, между ними — плавный переход.

Окружность S^1 : определение

Определение (Окружность как HIT)

```
data S1 : Type where  
  base : S1      -- point  
  loop : base = base -- loop
```



Два ингредиента

- 1 Точка `base` — «базовая точка»
- 2 Путь `loop : base = base` — «петля»

Окружность: интуиция

Аналогия: резинка для волос



Берём интервал и склеиваем концы $\rightarrow$ получаем окружность.

Разница с интервалом

- Интервал: путь между разными точками 0_I и 1_I
- Окружность: путь из точки в саму себя ($\text{base} = \text{base}$)

Окружность: что порождает loop?

Петля — это генератор

Из одной петли loop по правилам группоида получаем:



Ключевой факт

Все пути $\text{base} = \text{base}$ в $\mathbb{S}^1$ — это loop^n для $n \in \mathbb{Z}$:

$$(\text{base} =_{\mathbb{S}^1} \text{base}) \simeq \mathbb{Z}$$

Число n — «сколько раз обошли» (со знаком направления).

Принцип рекурсии для S^1

Рекурсия по окружности

Чтобы задать функцию $f : S^1 \rightarrow X$, достаточно указать:

- 1 $x_0 : X$ — образ точки base
- 2 $p : x_0 = x_0$ — образ петли loop

$$S1\text{-rec} : \prod_{X:\text{Type}} \prod_{x_0:X} \prod_{p:x_0=x_0} (S^1 \rightarrow X)$$

Вычислительные правила

- $f(\text{base}) \equiv x_0$
- $\text{ap}_f(\text{loop}) = p$ (путь в X)

Категорный взгляд

S^1 — «инициальный объект» среди пар (точка + её автоморфизм).

Фундаментальная группа $\pi_1(\mathbb{S}^1) \simeq \mathbb{Z}$

Классический результат алгебраической топологии

Фундаментальная группа окружности — группа целых чисел.

В HoTT

- $\pi_1(\mathbb{S}^1)$ = группа путей $\text{base} = \text{base}$ с операцией конкатенации
- Используя HIT-определение $\mathbb{S}^1$ и UA, можно доказать:

$$\Omega(\mathbb{S}^1) := (\text{base} =_{\mathbb{S}^1} \text{base}) \simeq \mathbb{Z}$$

Значимость

Алгебраическая топология может быть формализована внутри теории типов.
Это не просто аналогия — это честное доказательство.

Пример: сфера $\mathbb{S}^2$

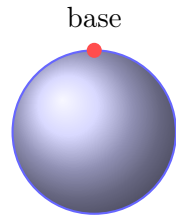
Сфера как HIT

- $\text{base} : \mathbb{S}^2$ — единственная точка
- $\text{surf} : \text{refl}_{\text{base}} = \text{refl}_{\text{base}}$ — 2-путь

Конструктор surf — это путь между путями.

Интуиция

- Точка = северный полюс
- 2-путь = «натяннутая поверхность»
- Нет нетривиальных 1-путей
- Есть нетривиальный 2-путь



$\text{surf} : \text{refl} = \text{refl}$

В HoTT доказывается: $\pi_2(\mathbb{S}^2) \simeq \mathbb{Z}$ (классический результат алгебраической топологии).

Quotient types через HIT

Проблема: фактор-типы

A/R — тип, где эквивалентные по R отождествлены. В MLTT: нетривиально.

Решение через HIT

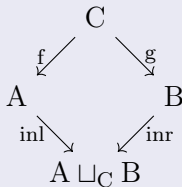
Point: $[_] : A \rightarrow A/R$; Path: $\text{eq} : R(a, b) \rightarrow [a] = [b]$

Пример: $\mathbb{Z}$ как $\mathbb{N} \times \mathbb{N} / \sim$

$(a, b) \sim (c, d) \Leftrightarrow a + d = b + c$. Число (a, b) представляет $a - b$.

Pushout как HIT

Диаграмма



Как HIT

```
data Pushout (f:C→A) (g:C→B) where
  inl : A → Pushout f g
  inr : B → Pushout f g
  push : (c:C) → inl(f c) = inr(g c)
```

Интуиция и примеры

Элементы A и B берём как есть, а элементы C «склеиваем» путями. Примеры: склейка интервалов $\rightarrow \mathbb{S}^1$; склейка границы диска $\rightarrow \mathbb{S}^2$.

Truncations как HIT

Определение (Propositional truncation)

$\|A\|_{-1}$ — превращает A в mere proposition:

```
data  $\|A\|_{-1}$  : Type where  
  |_ :  $A \rightarrow \|A\|_{-1}$   
  trunc : ( $x\ y : \|A\|_{-1}$ )  $\rightarrow x = y$ 
```

Определение (Set truncation)

$\|A\|_0$ — делает A h-set: добавляем path-constructor, убивающий пути между путями.

Интуиция

$\|A\|_{-1}$: « A обитаем» (без информации каким элементом). $\|A\|_0$: A как множество.

Связь truncations с Prop/Set в Coq/Agda

Системы типа Coq

- Prop — примерно «мир propositions» ($\text{h-level} \leq -1$)
- Set/Type — более общие типы
- Разделение «захардкожено» в системе

В HoTT

- Вместо ad-hoc Prop/Set — truncations как конструкции
- $\|A\|_{-1} \approx$ «A в Prop»
- $\text{isSet}(A)$ — свойство, которое можно проверять

Преимущество HoTT

Унивалентный мир даёт концептуально цельную картину.
Конкретные системы реализуют части этих идей.

План лекции

- 1 Рекап: Id-типы и h-уровни
- 2 Эквивалентности типов
- 3 Аксиома унивалентности
- 4 Высшие индуктивные типы (HIT)
- 5 Модели и Cubical Type Theory
- 6 Заключение

Проблемы с UA и классическими моделями

Классические модели MLTT

- Теоретико-множественные интерпретации
- Доменные модели
- Категорные модели (локально декартово замкнутые категории)

Проблема с UA

UA требует: $A = B \Leftrightarrow \text{Equiv}(A, B)$

В простой «модели во множествах» это не работает:

- Две разные реализации группы с одинаковой структурой
- В set-теории они не равны, но изоморфны
- UA требует их отождествить

Решение: модели HoTT живут в мире ∞ -категорий, [simplicial sets](#) и т.д.

Каноничность и вычислимость

Свойство каноничности

«Любой замкнутый терм типа $\mathbb{N}$ редуцируется к `zero` или `succ(...succ(zero)...)»`

Проблема с UA как аксиомой

Если просто взять $\text{MLTT} + \text{UA}$ как аксиому:

- Неочевидно, сохраняется ли каноничность
- `transport` по `ua(e)` «застревает» — не вычисляется
- Может быть сломана нормализация

Мотивация для новых систем

Нужна теория типов, где UA не просто аксиома, а имеет вычислительные правила.

Cubical Type Theory: идея

Ключевая идея cubical подхода

Вводим явный «интервал» $\mathbb{I}$ как примитив теории.

Путь $p : a =_A b$ представляется как функция:

$$p : \mathbb{I} \rightarrow A$$

с условиями:

$$p(0) \equiv a, \quad p(1) \equiv b$$

Что это даёт?

- Пути — конструктивные объекты
- По ним можно вычислять
- `transport`, `UA`, `HT` получают редукционные правила

Интервал $\mathbb{I}$ в cubical

Примитивный интервал

$\mathbb{I}$ — специальный тип с двумя элементами $0, 1 : \mathbb{I}$

Однако $\mathbb{I}$ не является обычным типом — это «pretype» со специальными правилами.

Путь как функция

$$\text{Path } A \ a \ b := (i : \mathbb{I}) \rightarrow A \quad \text{с} \quad p(0) \equiv a, \ p(1) \equiv b$$

Путь — это буквально функция из интервала с фиксированными концами.

Пример: refl

$\text{refl}_a := \lambda i. a$ — константная функция, $(\lambda i. a)(0) \equiv a \equiv (\lambda i. a)(1)$

Вычисления с путями

$p(i)$ для конкретного i вычислимо — можно «пройти» по пути.

Кан-операции в cubical

Проблема: заполнение кубов

Имея части куба, нужно построить недостающую грань.
Это обобщение `transport` и `composition` путей.

Операция `hcomp`

Композиция путей $p \cdot q$ как заполнение квадрата (homogeneous composition).

Операция `transp`

$\text{transp}(\lambda i. A(i)) a_0 : A(1)$ — перенос по семейству типов.

Ключевое свойство: эти операции вычисляются.

UA и HIT в cubical мире

Cubical Agda / cubicaltt

- UA реализована как конструктор с редукционными правилами
- $ua(e)$ — не «чёрный ящик», а вычисляемый терм
- $transport$ по $ua(e)$ редуцируется к применению e

Что восстанавливается?

- Нормализация — все термы имеют нормальную форму
- Каноничность — замкнутые термы типа $\mathbb{N}$ вычисляются
- Decidable type checking — проверка типов разрешима

С точки зрения PL

Cubical TT — один из путей сделать HoTT исполняемым языком программирования.

Где живёт HoTT/Cubical сегодня?

- Agda (cubical mode)
 - Полная поддержка HIT, UA с вычислительными правилами
 - Активно развивается, большая библиотека
- Coq + HoTT library
 - Унивалентные основы как библиотека (аксиоматически)
 - Экспериментальные cubical-расширения
- Lean
 - HoTT-проекты существуют
 - Но без полного cubical ядра
- Arend
 - Специально разработан для HoTT
 - Встроенная поддержка HIT и унивалентности

Пример: Cubical Agda

Определение окружности

```
data S1 : Type where
  base : S1
  loop : base = base
```

UA вычисляется

```
-- e : A  $\simeq$  B
-- ua e : A = B
-- transport (ua e) a
--   computes to (e .fst a)
```

Практическое значение

Можно запускать программы с UA; доказательства не застревают; HoTT — полноценный ЯП.

План лекции

- 1 Рекап: Id-типы и h-уровни
- 2 Эквивалентности типов
- 3 Аксиома унивалентности
- 4 Высшие индуктивные типы (HIT)
- 5 Модели и Cubical Type Theory
- 6 Заключение

Что мы узнали:

- Id-типы как типы путей
- h-уровни: `prop`, `set`, `groupoid`...
- Формальное определение `h-set`
- Эквивалентности типов
- Аксиома унивалентности (UA)
- Высшие индуктивные типы
- Cubical Type Theory

Ключевые идеи:

- Типы — это пространства
- Равенство — это путь
- UA: $\text{Equiv}(A, B) \simeq (A = B)$
- HIT: топология в типах
- Cubical: вычислимая UA

Инструменты:

- Cubical Agda
- HoTT library (Coq)
- Arend

Куда копать дальше?

Литература

- HoTT Book — «библия» HoTT, свободно доступна
- Introduction to Univalent Foundations (Rijke) — более современное изложение
- Cubical Agda Tutorial — практическое введение

Направления исследований

- Синтетическая алгебраическая топология — гомотопические группы, спектры
- Синтетическая теория категорий — ∞ -категории в HoTT
- Модальная HoTT — cohesion, differential cohomology
- Directed type theory — направленные пути, $(\infty, 1)$ -категории

Завершение курса

Что было в курсе

- 1 λ -исчисление: нетипизированное и типизированное
- 2 λ -куб: от простых типов к зависимым
- 3 MLTT: Π , Σ , Id-типы
- 4 CIC и Coq: практические системы
- 5 HoTT: унивалентность и высшие типы

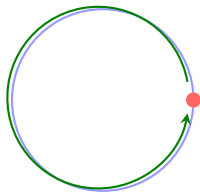
Главный вывод

Теория типов — это:

- Основание математики (альтернатива теории множеств)
- Язык программирования (Curry–Howard)
- Язык спецификаций и верификации
- Связь логики, математики и computer science

Спасибо за внимание

Вопросы?



$$\Omega(S^1) \simeq \mathbb{Z}$$